

Article

A Cascaded Batch Bayesian Yield Optimization Method for Analog Circuits via Deep Transfer Learning

Ziqi Wang ¹ , Kaisheng Sun ¹ and Xiao Shi ^{2,*}

¹ School of Integrated Circuits, Southeast University, Nanjing 210096, China; wangzq98@seu.edu.cn (Z.W.); sunkaisheng@seu.edu.cn (K.S.)

² School of Computer Science and Engineering, Southeast University, Nanjing 210096, China

* Correspondence: xshi@seu.edu.cn

Abstract

In nanometer integrated-circuit (IC) manufacturing, advanced technology scaling has intensified the effects of process variations on circuit reliability and performance. Random fluctuations in parameters such as threshold voltage, channel length, and oxide thickness further degrade design margins and increase the likelihood of functional failures. These variations often lead to rare circuit failure events, underscoring the importance of accurate yield estimation and robust design methodologies. Conventional Monte Carlo yield estimation is computationally infeasible as millions of simulations are required to capture failure events with extremely low probability. This paper presents a novel reliability-based circuit design optimization framework that leverages deep transfer learning to improve the efficiency of repeated yield analysis in optimization iterations. Based on pre-trained neural network models from prior design knowledge, we utilize model fine-tuning to accelerate importance sampling (IS) for yield estimation. To improve estimation accuracy, adversarial perturbations are introduced to calibrate uncertainty near the model decision boundary. Moreover, we propose a cascaded batch Bayesian optimization (CBBO) framework that incorporates a smart initialization strategy and a localized penalty mechanism, guiding the search process toward high-yield regions while satisfying nominal performance constraints. Experimental validation on SRAM circuits and amplifiers reveals that CBBO achieves a computational speedup of $2.02\times$ – $4.63\times$ over state-of-the-art (SOTA) methods, without compromising accuracy and robustness.

Keywords: process variation; transistor fault; importance sampling; Bayesian optimization



Academic Editor: Costas Psychalinos

Received: 8 January 2026

Revised: 21 January 2026

Accepted: 22 January 2026

Published: 25 January 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

As technology nodes scale into the nanometer regime, random process variations have emerged as a critical challenge in integrated-circuit (IC) design [1]. In metal-oxide-semiconductor field-effect transistors (MOSFETs), variations in process parameters such as threshold voltage (V_{th}) mismatches induce significant fluctuations in circuit performance. This effect is especially critical in highly sensitive circuits such as SRAM (static random-access memory) blocks and analog circuits [2]. These device-level variations can manifest as latent parametric faults, which often evade detection under nominal operating conditions. Consequently, they may trigger startup failures during read/write access or induce hold instability, ultimately compromising circuit reliability. Notably, such performance degradation will most likely lead to manufacturing yield loss, posing substantial challenges for cost-effective IC fabrication [3].

Manufacturing yield loss is defined as the proportion of fabricated ICs that fail to satisfy predefined performance or functional specifications. In advanced nodes, yield-limiting defects primarily originate from two distinct sources—systematic and random. Systematic defects, such as those induced by lithographic proximity effects, exhibit strong layout dependency and spatial correlation across wafers [4]. In contrast, random process variations have statistical properties and arise from processes that are orthogonal to design implementation. The primary contributors to process variations include RDF (random dopant fluctuation) and LER (line edge roughness) [5]. Parametric yield loss results from random process-induced variations that degrade circuit performance and reliability beyond acceptable design thresholds. Therefore, accurately estimating the fault probability induced by process variations in the early stages of circuit design is crucial for ensuring system-level circuit reliability.

The Monte Carlo (MC) simulation is regarded as the gold standard for parametric yield and reliability analysis [6,7]. For instance, to provide the failure probability in the range of 10^{-5} with an accuracy of $\pm 10\%$ with 95% confidence level, MC typically requires more than 10^7 transistor-level simulations. This is computationally impractical for large-scale circuits and industrial designs. The scalability limitations of conventional MC methods are particularly evident in high-sigma and high-dimensional parameter spaces.

In advanced technology nodes, circuit designers must not only ensure functional performance but also navigate complex trade-offs among performance, power, area, and reliability [8–10]. As a result, reliability-driven design optimization is increasingly shifting to the schematic stage, where it forms part of an extended design metric known as PPAY (performance, power, area, yield) [11]. This shift reflects a growing emphasis on circuit reliability in modern electronic design automation (EDA) flows. Existing reliability and yield optimization problems still suffer from massive simulation overhead and expensive iterative redesign cycles. Thus, developing efficient yield analysis and optimization techniques is essential for enhancing the robustness and competitiveness of customized circuits.

Importance sampling (IS) techniques [12–16] have become the mainstream yield analysis methods that shift the sampling distribution to sample failure regions with significantly higher probability. The precision and efficiency of IS methods depend on the selection of optimal shift vectors (OSVs) to a great extent. On this basis, Ref. [17] proposes an all-sensitivity adversarial importance sampling (ASAIS) yield optimization method, which transforms the optimization problem into maximizing the ℓ_2 -norm of the OSVs and solves the optimization target based on sensitivity analysis. However, the main drawback of these IS methods is that the computational efficiency of searching OSVs decreases exponentially with circuit dimensions. Surrogate models such as radial basis network, and tensor approximation [18–20] can build the nonlinear mapping between circuit performance and variation parameters with negligible elapsed time to enhance the evaluation efficiency of yield estimations, which generally requires massive training samples to guarantee accuracy.

Since yield analysis methods are highly time-consuming and inevitably produce estimation errors, Bayesian optimization (BO)-based methods [21] are introduced as a non-parametric optimization method to the yield optimization region, which can derive a quasi-optimal solution under the evaluation cost of seldom. Ref. [22] proposes an incremental BO framework with the weighted expected improvement (wEI) criterion, which utilizes the different incumbents in low and high-accuracy stages to guide the optimization process. Moreover, a freeze–thaw BO (FTBO) method [23] temporarily freezes the yield analysis for certain designs with low yield estimation, and gradually improves analysis accuracy in areas with high yield to enhance efficiency. Although academics have proposed various BO frameworks to guide efficient global search, the simulation cost per iteration remains a significant obstacle to mainstream yield optimization work.

Figure 1 illustrates how yield estimation results are iteratively integrated, guiding the selection of the next design sample using the acquisition strategy. This similarity between excessive yield estimation iterations inspires us to apply the transfer learning (TL) concept to the yield optimization problem. The basic idea of TL is to solve the regression or classification problem of the target domain based on the knowledge of the source domain. Thus, we can infer information about unknown design parameter samples based on the existing knowledge of the pre-trained models [24].

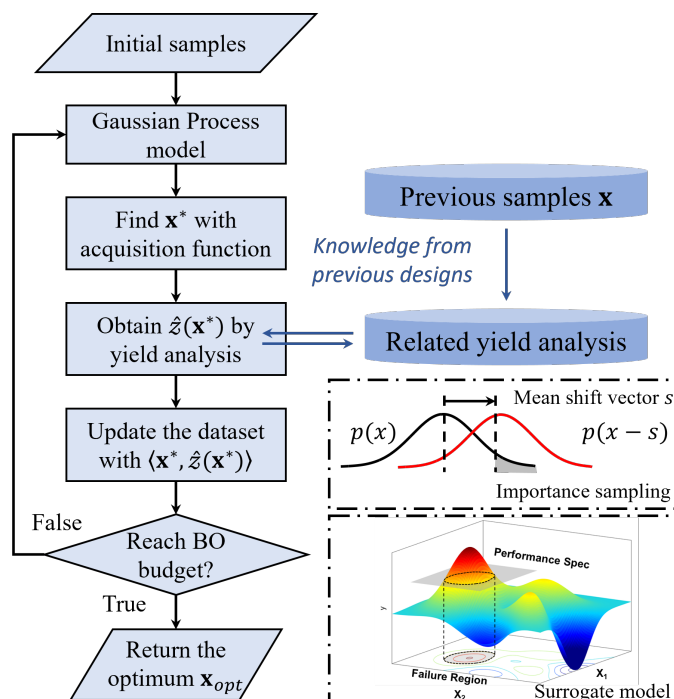


Figure 1. Schematic of the conventional BO framework for yield optimization. To save simulation costs, we can leverage prior design knowledge through pre-trained models to obtain current yield observations.

The key contributions of this paper include the following:

- We first propose a Deep Transfer Learning-assisted IS method (DTL-IS) that leverages network predictions to identify failure region boundaries accurately. By employing pre-trained neural network models, DTL-IS develops a knowledge transfer mechanism guided by a hybrid loss function to enable effective fine-tuning under specific circuit designs. Moreover, an adversarial perturbation vector is incorporated to calibrate model outputs and enhance prediction robustness. DTL-IS implements failure probability estimation using the distorted sampling distribution, which reduces the overall simulation cost for repetitive yield analysis.
- We further develop a cascaded batch Bayesian optimization framework, termed CBBO, which enhances computational efficiency by leveraging sparse matrix operations and flexible anisotropic kernels. Inheriting the promising samples obtained from the model uncertainty, CBBO proposes a penalization-improvement mechanism that excludes previously observed points during batch selection, thereby promoting sample diversity and encouraging exploration.
- The overall optimization framework is validated using SRAM circuits and analog circuits. Experimental results show that CBBO achieves a computational speedup ranging from 2.02× to 4.63× compared to state-of-the-art (SOTA) optimization methods. CBBO can maintain comparable prediction accuracy and effectively improve circuit reliability in multiple circuit designs.

The remainder of this article is organized as follows: In Section 2, we present the problem formulation and the principle of the BO method. The detailed TL strategy and yield analysis method DTL-IS is illustrated in Section 3. The optimization approach based on a cascaded mechanism is depicted in Section 4. Experimental results validate the efficiency of the proposed method in Section 5. Finally, we summarize the whole framework and effects in Section 6.

2. Preliminaries

2.1. Problem Formulation

Reliability quantifies the probability that a system will perform its intended function under specified operational conditions for a designated duration without failure. Statistical reliability analysis provides a probabilistic framework for assessing this reliability. Within this framework, uncertain system parameters are modeled as random variables $\mathbf{v}$, characterized by a joint probability density function (PDF) $f(\mathbf{v})$. A limit state function defines the system's performance boundary as follows:

$$g(\mathbf{v}) = \begin{cases} > 0, & \text{safe domain,} \\ \leq 0, & \text{failure domain.} \end{cases} \quad (1)$$

The system-level failure probability is given as follows:

$$P_f = P[g(\mathbf{v}) \leq 0] = \int_{\{g(\mathbf{v}) \leq 0\}} f(\mathbf{v}) \, d\mathbf{v} \quad (2)$$

Statistical reliability analysis involves quantifying the uncertainty of input parameters, evaluating the failure probability P_f through methods such as Monte Carlo simulation, importance sampling, or subset simulation, and iteratively refining the design to drive P_f below a prescribed target P_f^* . Only when $P_f \leq P_f^*$ can the system be certified to meet reliability requirements.

In the IC design field, statistical reliability challenges arise from inevitable random variations inherent in semiconductor fabrication processes, primarily attributable to RDF and LER. These variations induce statistical deviations in critical device parameters such as threshold voltage (V_{th}), gate oxide thickness, and effective channel length. As device scaling reaches deep nanometer regions, these parameter variations become increasingly pronounced, significantly exacerbating performance variability and compromising circuit-level reliability.

As shown in Figure 2, let $\mathbf{x} = [x_1, x_2, \dots, x_{d_x}]^T$ denote a d_x -dimensional vector representing all relevant design parameters extracted from the netlist, such as transistor widths and lengths. Each design parameter x_i is specified by circuit designers in advance and takes on a set of discrete values within the range $[l_i, u_i]$. Correspondingly, the random process variable can be denoted as $\mathbf{v} = [v_1, v_2, \dots, v_{d_v}]^T$, such as threshold voltage and gate equivalent oxide thickness. The statistical properties of v are typically provided by the semiconductor foundry through a device model file. In practice, these process parameters are often modeled as independent Gaussian random variables as follows:

$$p(v_i) = \frac{1}{\sqrt{2\pi}\sigma_i} \exp\left(-\frac{(v_i - \mu_i)^2}{2\sigma_i^2}\right), \quad (3)$$

where μ_i and σ_i denote the mean and standard deviation of parameter v_i , respectively. Moreover, the distribution of process parameters is assumed to be independent of the design parameters $\mathbf{x}$.

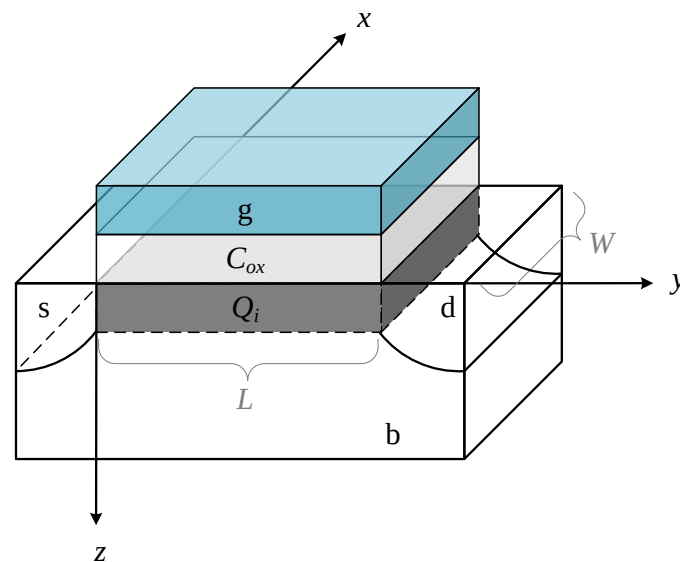


Figure 2. The simplified schematic of the MOSFET structure.

The statistical distribution of circuit performance y is affected by design and process parameters together. The mapping relationship can be denoted as $y = f(\mathbf{x}, \mathbf{v})$, which can be derived from transistor-level HSPICE simulations. Ultimately, this nonlinear mapping transforms process parameter variations into circuit-level reliability risks, necessitating statistical circuit optimization. The circuit failure event occurs when the circuit performance y does not meet the specified requirements. We introduce the indicator function $I(\mathbf{x}, \mathbf{v})$ to identify pass/fail of a variation sample:

$$I(\mathbf{x}, \mathbf{v}) = \begin{cases} 0, & \text{if } \mathbf{v} \notin S; \\ 1, & \text{if } \mathbf{v} \in S, \end{cases} \quad (4)$$

where S denotes the failure region in the process parameter space. The indicator function $I(\mathbf{x}, \mathbf{v})$ can be interpreted as a failure detector, identifying whether a given process sample results in failure. Given specific design parameters $\mathbf{x}$ and process parameters $\mathbf{v}$, the corresponding circuit failure probability P_{fail} can be calculated as

$$P_{fail} = \int_V I(\mathbf{x}, \mathbf{v}) p(\mathbf{v}) d\mathbf{v}, \quad (5)$$

where $p(\mathbf{v})$ is the joint probability density function of the process parameters. Improving the reliability and resistance to process variations of circuits can be viewed as minimizing the failure probability P_{fail} .

It should be noted that the integral is unfeasible to solve because the term $I(\mathbf{x}, \mathbf{v})$ has no analytical expressions. The most straightforward approach is the Monte Carlo (MC) method, which involves randomly sampling N independent samples $\{\mathbf{v}_i\}_{i=1}^N$ from the distribution $p(\mathbf{v})$. Based on transistor-level simulations, the proportion of failure samples provides an unbiased estimator of the failure probability as follows:

$$\hat{P}_{MC} = \frac{1}{N} \sum_{i=1}^N I(\mathbf{x}, \mathbf{v}_i), \quad (6)$$

which converges to P_{fail} when a sufficient large sample set is given [14]. However, brutal MC methods encounter failure probability assessment bottlenecks, as the sampling efficiency for failure samples is extremely low when the failure probability is below 10^{-6} .

To address this issue, IS-based methods propose a shifted sampling distribution $q(\mathbf{v})$ to improve the sampling efficiency of failure samples. The failure rate evaluated by IS can be expressed as follows:

$$\begin{aligned} P_{IS}(\mathbf{x}) &= \int_V I(\mathbf{x}, \mathbf{v}) \cdot \frac{p(\mathbf{v})}{q(\mathbf{v})} \cdot q(\mathbf{v}) d\mathbf{v} \\ &= \int_V I(\mathbf{x}, \mathbf{v}) \cdot w(\mathbf{v}) \cdot q(\mathbf{v}) d\mathbf{v}, \end{aligned} \quad (7)$$

where the $w(\mathbf{v})$ denotes the importance weights. $q(\mathbf{v})$ can be represented as $q(\mathbf{v}) = p(\mathbf{v} - \vec{\mu}_q)$, where $\vec{\mu}_q$ denotes the optimal shift vector (OSV) from the original distribution. The implementation of $\vec{\mu}_q$ can assist in identifying the boundary of the failure region, thereby enhancing the efficiency of collecting failure samples. Generating samples from $q(\mathbf{v})$, then the failure rate evaluated by IS is

$$\hat{P}_{IS}(\mathbf{x}) = \frac{1}{N} \cdot \sum_{i=1}^N I(\mathbf{x}, \mathbf{v}_i) w(\mathbf{v}_i). \quad (8)$$

Moreover, the variance of IS estimator $\sigma(\hat{P}_{IS})$ can be analytically expressed as follows:

$$\sigma(\hat{P}_{IS}) = \frac{1}{N^2} \cdot \left[\sum_{i=1}^N \left(\frac{p(\mathbf{v}_i)}{q(\mathbf{v}_i)} \right)^2 I(\mathbf{v}_i) - N \cdot \hat{P}_{IS}^2 \right], \quad (9)$$

which serves as a quantitative measure of the estimation accuracy and numerical stability. A smaller variance indicates higher confidence in the failure probability estimation, reflecting the effectiveness of shifted sampling distribution and enhanced computational efficiency.

In the schematic design phase, the yield optimization problem aims to find an optimal design solution $\mathbf{X}^*$ that exhibits minimal sensitivity to circuit performance degradation caused by process variations. Taking the nominal performance specifications c_i as constraints, the optimization problem can be formulated as

$$\begin{aligned} \mathbf{x}^* &= \operatorname{argmin}_{\mathbf{x} \in X} P_{fail}, \\ \text{s.t. } \bar{y}_i(\mathbf{x}) &\leq c_i, i = 1, \dots, k, \end{aligned} \quad (10)$$

where $\bar{y}_i(\mathbf{x})$ denotes the i -th nominal performance of design $\mathbf{x}$. The yield optimization process is limited to the feasible domain that meets the required specifications with constraints $\bar{y}_i(\mathbf{x}) \leq c_i$. The main challenges in solving optimization problems are the high simulation overheads and the complexity of the objective function. Firstly, the entire yield optimization iteration process requires significant circuit simulation overheads, especially for high-dimensional scenarios. Additionally, traditional optimization methods like gradient descent-based algorithms are not available due to the complexity of the objective function.

2.2. Bayesian Optimization via Gaussian Process Model

The BO framework embodies the philosophy behind the Bayesian theorem, where the probabilistic model is updated under the observation of new samples obtained guided by the acquisition function. Gaussian process (GP) regression is a broad choice for the probabilistic model due to its model flexibility and uncertainty quantification [25]. Considering a data set containing of n input samples $X = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n]^T$ and observations $Z = [\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_n]^T$ with observation noise $\epsilon \sim \mathcal{N}(0, \sigma_\epsilon^2)$, the prior distribution of unseen function can be denoted as $z(\mathbf{x}) \sim \mathcal{GP}(\mu, \Sigma)$, where the μ and Σ represent the mean and positive definite covariance function respectively. The mean function is often set to zero after data normalization.

To describe properties related to the parameter space, the radial basis function (RBF) kernel is employed for selected elements $\{\mathbf{x}, \mathbf{x}'\}$ in Σ

$$\kappa_{\text{RBF}}(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{1}{2}(\mathbf{x} - \mathbf{x}')^\top \Theta^{-2}(\mathbf{x} - \mathbf{x}')\right), \quad (11)$$

where Θ is a vector containing length scales θ_d for each dimension. Given a new sample $\mathbf{X}_*$, the joint Gaussian distribution of the observed outputs Z and the predicted output $\mathbf{z}_*$

$$\begin{bmatrix} Z \\ \mathbf{z}_* \end{bmatrix} \sim \mathcal{N}\left(0, \begin{bmatrix} K(X, X) + \sigma_\epsilon^2 I & K(X, \mathbf{x}_*) \\ K(X, \mathbf{x}_*)^\top & \kappa(\mathbf{x}_*, \mathbf{x}_*) \end{bmatrix}\right), \quad (12)$$

where the $K(X, X) = [\kappa(\mathbf{x}_i, \mathbf{x}_j)]_{\mathbf{x}_i, \mathbf{x}_j \in X}$ denotes the covariance matrix in X , and $K(X, \mathbf{x}_*) = [\kappa(\mathbf{x}_i, \mathbf{x}_*)]_{\mathbf{x}_i \in X}$.

The acquisition function leverages posterior probability to guide the next point search. The next query point is selected by maximizing the acquisition function value $\alpha_n(\mathbf{x}; \{\mathbf{x}_i, z_i\}_{i=1}^n)$ within the search space $\mathcal{D}$

$$\mathbf{x}_{\text{next}} = \arg \max_{\mathbf{x} \in \mathcal{D}} \alpha_n(\mathbf{x}; \{\mathbf{x}_i, z_i\}_{i=1}^n). \quad (13)$$

A widely used acquisition function is the Expectation Improvement (EI) [26] strategy. The EI strategy of prediction uncertainty and improvement quantity aims to find the sample that provides the greatest improvement in the expectation. As for the incumbent ζ which is overwhelmingly set as the current best output, the improvement of yield observation under candidate sample point $\hat{z}(\mathbf{x})$ is defined as $I(\hat{z}(\mathbf{x})) = \max(0, \hat{z}(\mathbf{x}) - \zeta)$. Thus EI strategy is expressed as

$$\begin{aligned} \text{EI}(\mathbf{x}) &= \mathbb{E}[I(\hat{z}(\mathbf{x}))] = (\zeta - \mu(\mathbf{x}))\Phi(\lambda) + \sigma(\mathbf{x})\phi(\lambda) \\ \lambda &= (\zeta - \mu(\mathbf{x}))/\sigma(\mathbf{x}) \end{aligned} \quad (14)$$

where $\mu(\mathbf{x})$ and $\sigma(\mathbf{x})$ are the posterior mean and variance of $\hat{z}(\mathbf{x})$, $\Phi(\cdot)$ and $\phi(\cdot)$ denote the CDF and PDF of the standard Gaussian distribution respectively. EI keeps the balance between local exploitation of present best solutions and global exploration of other promising regions by adjusting ζ .

As for the constrained yield optimization problem shown in Equation (10), the wEI method solved by multiple starting points (MSPs) proposed in [22] is proven to be effective, which takes the form of

$$w\text{EI}(\mathbf{x}) = PF(\mathbf{x}) \cdot \text{EI}(\mathbf{x}), \quad (15)$$

Here, $PF(\mathbf{x}) = \prod_{i=1}^k P(\bar{y}_i(\mathbf{x}) \leq c_i)$ represents the probability of all nominal constraints being satisfied, and c_i denotes the nominal threshold for the i -th performance metric. The wEI(X) is viewed as the form of EI(X) weighted by the probability of meeting nominal design specifications. The wEI method can significantly improve the effectiveness of the constrained yield optimization and locate the feasible regions [27].

3. Yield Analysis via Deep Transfer Learning

In this section, we present the details of the proposed yield analysis method DTL-IS, which utilizes a CNN classifier prediction to determine the OSVs of IS yield estimation. For the failure probability estimation $P_{\text{fail}}(\mathbf{x})$ of a specific set of circuit design parameters X , we develop a hybrid loss function to fine-tune the pre-trained network models. Then, DTL-IS collects adversarial samples to obtain the calibrated model predictions and OSVs. The transfer mechanism enables efficient yield analysis of unseen design samples X without

heavy simulation overhead. We implement the training and fine-tuning of networks based on PyTorch library (<https://pytorch.org>).

3.1. Adaptive Sampling with CNN Classifier

In order to search for the OSVs, we first initialize the dataset by pre-training the CNN model as a failure sample discriminator. As for the preparation of the training set, we first randomly select multiple $\{X_i^s\}_{i=1}^L$ to avoid the accuracy loss caused by reusing knowledge. In the model training stage, we improve the sampling strategy from the parametric space V to alleviate the effect of imbalanced training data [28] on the surrogate model accuracy. Using the adaptive synthetic sampling (ADASYN) method, the synthesized samples $\mathbf{v}_{syn}$ are generated as pseudo failed samples using the centroid of each failure cluster $\mathbf{v}_{f,c}$ as

$$\mathbf{v}_{syn} = \mathbf{v}_{f,c} + (\mathbf{v}_n - \mathbf{v}_{f,c}) \times \lambda, \quad (16)$$

where $\mathbf{v}_n$ represents the nearest sample of $\mathbf{v}_{f,c}$ in each failure region. The class imbalance ratio of passed and failed samples should meet the maximum tolerance $d_{th} = 0.6$ to determine the number of pseudo samples. The generated samples will be merged into the training set $\mathcal{T}_i, i = 1, 2, \dots, L$ for each X , which helps to obtain broader trust region intervals.

In Table 1, after data normalization and preprocessing, the m training samples $m \times d_v$ are first handled by the convolutional layer to extract parameter features. It should be highlighted that zero paddings are adapted to maintain the dimensionality of features during the convolution process. The previous layer's outputs are convolved between the convolution layers with several convolutional kernels to be updated in the training process. The output of the convolutional operators, combined with the bias and the ReLU activation function, forms the feature mapping of the next layer. Moreover, to alleviate the covariate shift effects due to training set generation methods, we add the batch normalization (BN) layers [29] to standardize the data features and enhance the numerical representations.

Table 1. Structure of the CNN classifier for failure probability analysis.

Layer Name	Output Dimensions	Effects
Input Layer	$1000 \times d_v$	Normalized design parameters.
Convolutional Layer 1	$1000 \times d_v \times n1$	Feature extraction.
Convolutional Layer 2	$1000 \times d_v \times n2$	Feature extraction.
Batch Normalization 1	$1000 \times d_v \times n2$	Stabilize covariate shift.
Pooling Layer 1	$500 \times d_v \times n2$	Reduces complexity.
Convolutional Layer 3	$500 \times d_v \times n3$	Complex feature extraction.
Fully Connected Layer 1	n_fc1	Connects flattened feature.
Fully Connected Layer 2	n_fc2	Further processing.
Output Layer	1	Binary classification.

Except for the convolution module, three fully connected layers with a sigmoid activation function are employed to implement classification tasks. The classification loss function adopts cross-entropy loss $\mathcal{L}_{cls}$ which can be denoted as

$$\mathcal{L}_{cls} = -(I(\mathbf{v}) \cdot \log(\mathbf{p}) + (1 - I(\mathbf{v})) \cdot \log(1 - \mathbf{p})), \quad (17)$$

where $\mathbf{p}$ represents the positive predictive probability of each sample, and $I(\mathbf{v})$ represents the truth label of each training sample. In the detailed experiments of training models, the neural network is trained with the following parameter settings: *input size* = $1000 \times d_v$

(d_v is related to specific circuit parameters), *batch size* = 20, *initial learning rate* = 0.0004 (decay ten times for every 200 epochs).

3.2. Multi-Source Transfer Learning

The intuitive concept is to regard the initial sample set as the source domain and leverage domain-specific knowledge to perform yield analysis for each unseen sample. Specifically, the knowledge transfer concept is founded on the similarity and transferability between the performance distributions of different design parameters $\mathbf{x}$. Based on the assumption that different design samples share the same process variation space, we can view the performance distribution discrepancy between two design samples as conditional shifts according to the transfer learning concept. Specifically, the distribution discrepancy between the source and target domains can be denoted as

$$p(y^s | \mathbf{v}) \neq p(y^t | \mathbf{v}). \quad (18)$$

This motivates us to learn the unknown performance distribution from previous knowledge via transfer learning. Thus, we develop the following multi-source transfer learning method founded on the idea of semi-supervised domain adaptation [30,31], which evolves limited labeled samples.

Poorly chosen transfer strategies or significant feature discrepancies can result in negative transfer effects, leading to degraded model performance and inaccurate yield predictions [32]. Choosing a source domain with high similarity to the target domain can effectively improve the transfer performance. With domain-invariant representations from source models, the similarity can be quantified as the Euclidean distance between the centroids of the failure regions from k-means clustering. Therefore, the similarity between two design samples $\mathcal{D}(\mathbf{x}_1, \mathbf{x}_2)$ can be regarded as the maximum cosine similarity values between two centroids, $\mathbf{c}$:

$$\mathcal{S}(\mathbf{c}_i, \mathbf{c}_j) = \frac{\mathbf{c}_i \cdot \mathbf{c}_j}{\sqrt{\|\mathbf{c}_i\|^2 \cdot \|\mathbf{c}_j\|^2}}. \quad (19)$$

The square root term in $\mathcal{S}(\mathbf{c}_i, \mathbf{c}_j)$ encodes distance information, helping to distinguish between design samples.

A key component of the multi-source transfer learning strategy is the design of a hybrid loss function to improve the estimation accuracy. Inheriting the weight coefficients of the convolutional module [33], to measure inter-domain distribution distance, the marginal distribution distance measure maximum mean discrepancy (MMD) is proposed to implicitly map the features into the Hilbert space and calculate the mean discrepancy. Considering the consistency of variation sample distributions $p(\mathbf{v}^s)$ and $p(\mathbf{v}^t)$ across different designs, the low value of MMD measurement results in poor transfer effectiveness. To quantify the classification discrepancy between different design samples from Equation (18), we reformulate a weighted version of MMD $\mathcal{L}_{wmmd}$ based on the class weights to align the discrepancy of feature distribution between each source domain and the target domain. Based on the feature mapping of the convolutional layer output, $\mathcal{L}_{wmmd}$ takes the following form:

$$\mathcal{L}_{wmmd} = \left\| \frac{1}{n_s} \sum_{i=1}^{n_s} \alpha_{s,i} \phi(\mathbf{v}_i) - \frac{1}{n_t} \sum_{j=1}^{n_t} \phi(\mathbf{v}_j) \right\|_{\mathcal{H}}^2, \quad (20)$$

where $\phi(\cdot)$ denotes the nonlinear mapping function to Hilbert space $\phi: \mathbb{R}^d \rightarrow \mathcal{H}$, and n_s and n_t represent the number of samples in source and target domains, respectively. To alle-

viate the effect of class weight bias, the auxiliary weight coefficients $\alpha_{s,c}$ can be calculated by the quantity ratio of labeled samples between source and target domains:

$$\alpha_{s,c} = n_t^c / n_s^c. \quad (21)$$

Thus, all samples of the source domain that belong to the same class will share the same $\alpha_{s,c}$ to match the target domain. For the semi-supervised domain adaptation in yield analysis, the weighted loss $\mathcal{L}_{wmmd}$ can accommodate the classification discrepancy and capture input features.

The whole transfer learning process is illustrated in Figure 3. We retain the pre-trained convolutional layers (w_c, b_c) to preserve feature extraction capabilities while fine-tuning the fully connected layers w_f according to the samples in the target domain. Next, we propose a hybrid loss function $\mathcal{L}_f$ to retrain the weights and biases, which consists of target domain classification loss $\mathcal{L}_{cls}$ and conditional distribution discrepancies loss. The hybrid function can be denoted as

$$\mathcal{L}_f = \mathcal{L}_{cls} + \lambda \mathcal{L}_{wmmd} + \beta \|w_f\|_{\ell_2}^2 \quad (22)$$

where λ and β are predefined trade-off coefficients. The $\mathcal{L}_f$ is employed to update the weights using backpropagation with Adam optimizer [34]. The training process continues until the $\mathcal{L}_f$ converges. To maintain the global properties of the target distribution, the weighted MMD loss term $\mathcal{L}_{wmmd}$ can measure the conditional distribution discrepancy between the source domain and target domain during the training process without introducing large computational overhead for large-scale training data. The predicted distribution will converge asymptotically in the transfer process to the unknown true distribution. The ℓ_2 regularization terms are introduced into the Adam optimizer to balance fitting training data and generalization of new data.

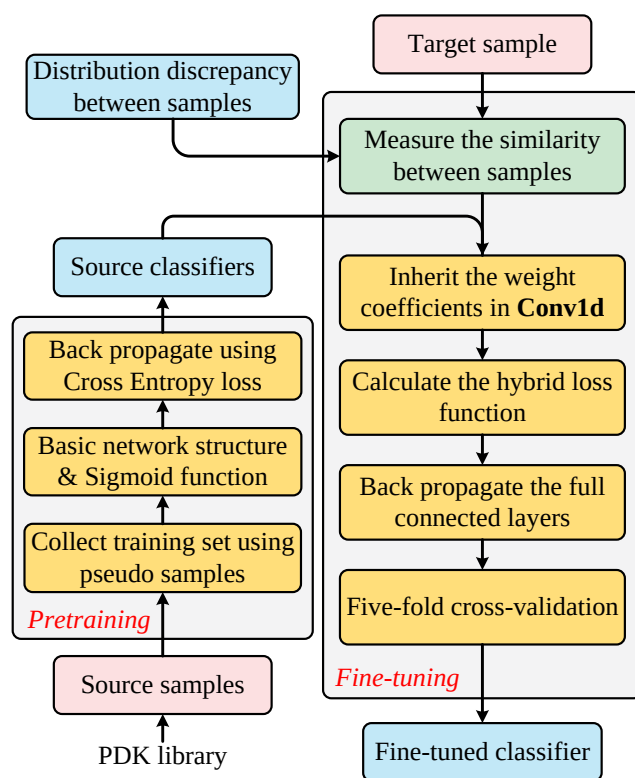


Figure 3. The schematic of the proposed TL method based on CNN, which adopts a hybrid loss function to fine-tune the model under conditional distribution shifts.

During the transfer stage, we utilize the coefficient of determination R-square (R^2) as the metric to evaluate the model estimation accuracy and degree of confidence. The R^2 can be denoted as

$$R^2 = 1 - \frac{\sum_{i=1}^n (I_i - \hat{I}_i)^2}{\sum_{i=1}^n (I_i - \bar{I})^2}, \quad (23)$$

where $\bar{I} = (1/n) \sum_{i=1}^n I_i$. The R^2 value ranges from 0 to 1, where a value closer to 1 indicates a better fitting effect of the model and determines the iteration of termination. To ensure the model generalizes well to unseen data, we employ a five-fold cross-validation approach, which ensures each sample is used for both training and validation. The cross-validation process helps to mitigate the risk of overfitting from the noise in the training data. In domain adaptation, cross-validation can help adjust model parameters that minimize distribution differences between the source and target domain.

The final classification performance can be comprehensively evaluated using the receiver operating characteristic (ROC) curves. As shown in Figure 4, we utilize different sizes of 6T SRAM bitcell and implement the training phase of classifiers. The data on the horizontal axis of the ROC curve is the false positive rate (FPR), which is the ratio of false positives to actual negatives. Meanwhile, the data on the vertical axis is the true positive rate (TPR), which is the ratio of true positives to actual positives. Moreover, the area under the curve (AUC) in the figure is the area enclosed by the ROC curve and the axes, and its value is proportional to the model's performance.

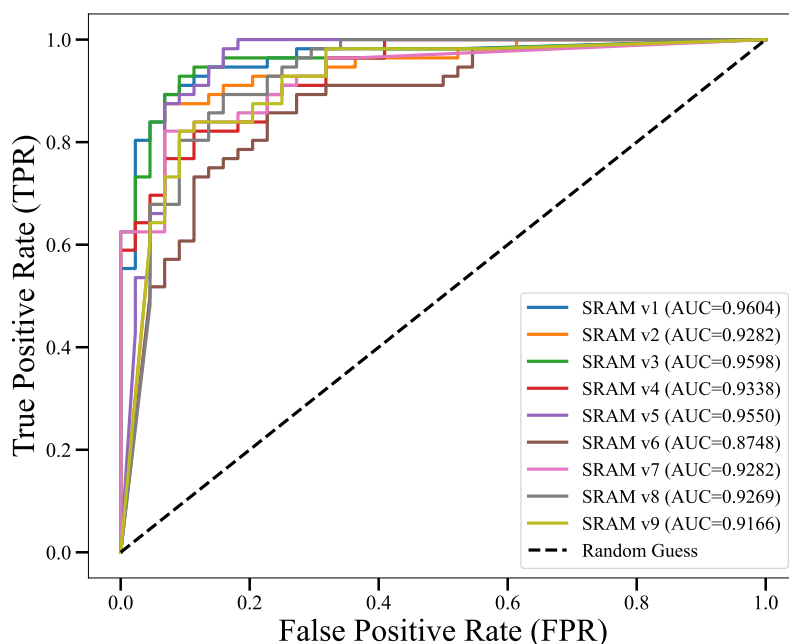


Figure 4. ROC curves of 6T SRAM bitcell with different sizes.

3.3. Failure Probability Estimation

Next, we implement the generation of adversarial vectors to calibrate the network predictions and obtain robust failure probability estimation. Based on the deep transfer classification model for a specific design sample $\mathbf{x}^t$, the efficiency of searching OSVs can be greatly improved by distinguishing between failed and successful samples for yield analysis. Searching OSVs can be viewed as an optimization problem, as

$$\begin{aligned} \vec{\mu}_* &= \arg \min \|\mathbf{v}\|^2 \\ \text{s.t. } I(\mathbf{x}, \mathbf{v}) &= 1, \end{aligned} \quad (24)$$

where the most probable failure point (MPFP) determines the selection of OSVs. With the identified failure region, the distorted distribution is constructed centering around it $q(\mathbf{v}) = p(\mathbf{v} - \bar{\boldsymbol{\mu}}_*)$. We employ a gradient-based deterministic method, SQP, to derive the local optimum of Equation (24).

However, OSV $\bar{\boldsymbol{\mu}}$ obtained by neural network predictions can easily lead to the final yield estimation distortion. Apart from network uncertainty from the model structure and the high-dimensional nature of parameter space, the inevitable misclassified samples and limited labeled samples may lead to significant bias in the decision boundary estimation [35]. Therefore, the ideal shifted vector $\bar{\boldsymbol{\mu}}_{true}$ can be represented using perturbation vector $\vec{r}$

$$\bar{\boldsymbol{\mu}}_{true} = \bar{\boldsymbol{\mu}} + \sum \vec{r}. \quad (25)$$

Specifically, after obtaining OSVs, we aim to calibrate the proposal distribution for each failure region. We consider that the CNN model's output probability $\mathbf{p}$ reflects the degree of estimation confidence and decision boundary. Meanwhile, the model prediction is relatively less robust with the sample $\mathbf{v}$ considered close to the decision boundary. As shown in Figure 5, starting from the current MPFP, an adversarial perturbation vector is generated along the opposite direction of the gradient of the cross-entropy loss $\mathcal{L}_{cls}$ as

$$\bar{\boldsymbol{\mu}}_{adv} = \bar{\boldsymbol{\mu}} + \varepsilon \cdot \text{sign}(\nabla_{\mathbf{v}} \mathcal{L}_{cls}). \quad (26)$$

During the perturbation, the gradient term $\nabla_{\mathbf{v}} \mathcal{L}_{cls}$ is calculated to identify samples for which the model is likely to be in error. While CNN provides a differentiable surrogate for classification, the final determination of failure or success is based on HSPICE simulations, which serve as the ground-truth oracle. The gradient is computed as

$$\text{sign}(\nabla_{\mathbf{v}} \mathcal{L}_{cls}) = \frac{\partial \mathcal{L}_{cls}(\mathbf{v} + \mathbf{r})}{\partial (\mathbf{v} + \mathbf{r})}. \quad (27)$$

An optimal adversarial vector $\bar{\boldsymbol{\mu}}_{adv}$ should be obtained through an iterative update approach, driving it closer to the ideal MPFP. To improve the stability and transferability of the optimization process, the learning rate ε is adaptively updated at each iteration based on the ℓ_2 -norm of the gradient as follows:

$$\varepsilon = \frac{1}{\|\nabla_{\mathbf{v}} \mathcal{L}_{cls}\|_2 + \delta}, \quad (28)$$

where δ is a small constant for numerical stability. The perturbation procedure terminates once the ℓ_2 -norm of $\bar{\boldsymbol{\mu}}_{adv}$ converges, indicating stability in the adversarial direction. By adjusting ε based on the ℓ_2 -norm, the method balances between too small and too large updates, ensuring stable convergence toward an optimal adversarial vector. Therefore, the method is robust to variations in the initial perturbation magnitude, provided the adaptive learning rate and the small constant δ are properly set.

During the iterative process, we aim to efficiently explore the failure region in the parameter space. To reflect the irregular geometry of the failure domain, we construct M proposal distributions centered around $\bar{\boldsymbol{\mu}}_*$, each designed to cover distinct sub-regions of the boundary. This multi-proposal approach enables better coverage of complex failure patterns that may not be well represented by a single distribution. After each IS iteration, the contribution of each sub-distribution to the failure probability estimation $\hat{P}_{fail}$ is evaluated. Based on the sampling effectiveness of each component, their mixture weights are adaptively updated to improve estimator efficiency and ensure faster convergence. The IS

estimator employs the relative standard deviation ρ of variance Equation (9) as the figure of merit (FoM) to characterize the confidence interval of estimation, which is defined as

$$\rho(\hat{P}_{fail}) = \frac{\sqrt{\sigma^2(\hat{P}_{fail})}}{\hat{P}_{fail}}. \quad (29)$$

FoM $\rho(\hat{P}_{fail})$ reflects the accuracy and confidence level of failure rate estimation. Here, we set the FoM threshold as $\rho_{th} = 0.1$, indicating that the estimation reaches a steady state with a confidence interval of 90%. The whole IS estimation flow is denoted in Algorithm 1. In the detailed experiments, HSPICE simulations are conducted in multi-process mode.

Algorithm 1 IS with adversarial perturbation.

Require: Original PDF of process parameters $p(\mathbf{v})$ for target sample $\mathbf{x}^t$, and FoM threshold ρ_{th} .
Ensure: Yield estimation $\hat{z}(\mathbf{x}^t)$.

- 1: Find initial OSVs $\vec{\mu}$ from fine-tuned network model;
 - 2: **while** not converge **do**
 - 3: $\vec{\mu} \leftarrow \vec{\mu} + \varepsilon \cdot \nabla_{\mathbf{v}} \mathcal{L}_{cls}$;
 - 4: **if** ℓ_2 -norm converge **then**
 - 5: $\vec{\mu}_* = \vec{\mu}$.
 - 6: **end if** ▷ OSV calibration using adversarial samples
 - 7: **end while**
 - 8: Generate M proposal distribution $\{q_j(\mathbf{v})\}_{j=1}^M$ around $\vec{\mu}_*$;
 - 9: Initial component weights $c_j = 1/M$;
 - 10: **while** $\rho(\hat{P}_{fail}) < \rho_{th}$ **do**
 - 11: Collect $N_j = c_j N$ samples under each proposal distribution $q(\mathbf{v})$;
 - 12: Compute weights $w_i \leftarrow p(\mathbf{v}_i) / \sum_{j=1}^M c_j q_j(\mathbf{v}_i)$;
 - 13: $\hat{P}_{fail} \leftarrow \sum_i w_i I(\mathbf{v}_i) / \sum_i w_i$; ▷ Mixture sampling distribution
 - 14: Compute variance $\{\rho_j\}_{j=1}^M$ for each component;
 - 15: Update component weights $c_j \leftarrow \rho_j / \sum_k \rho_k$;
 - 16: **end while**
 - 17: Yield estimation $\hat{z}(\mathbf{x}^t) = 1 - \hat{P}_{fail}$.
-

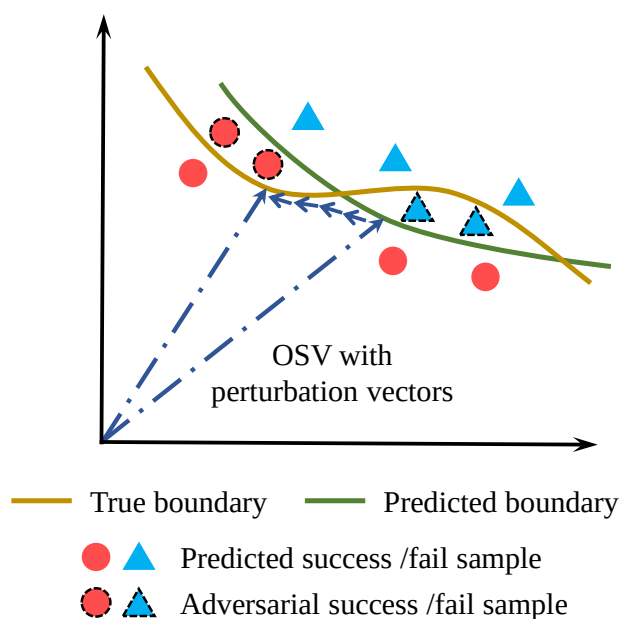


Figure 5. The schematic of the iterative calibration of OSVs using perturbation vectors. The arrows present the perturbation vectors.

4. Cascaded Yield Optimization Method

In this section, we employ the CBBO framework to implement yield optimization. From the estimation uncertainty of the sparse GP model, CBBO first generates multiple local optimums as promising samples in each iteration. Next, it collects batch samples around the regions of high acquisition values using the proposed local penalty function. This cascaded mechanism can facilitate efficient reliability-driven circuit design optimization with the constraints of nominal design specifications.

4.1. Global Exploration of Sparse GP Model

To alleviate the cold start issues resulting from an unfamiliar optimization function, we aim to enhance the exploration of promising samples from the probabilistic model during the optimization process. Conventional Bayesian optimization (BO) algorithms often face challenges in high-dimensional search spaces with intricate objective functions [36]. Without sufficient prior knowledge, these algorithms struggle with global exploration, requiring more optimization iterations to identify regions of interest effectively. As illustrated in the 2D yield optimization schematic Figure 6, the standard EI acquisition strategy tends to focus on global exploration in the initial stages, often without significant improvement in the best-observed values. This feature delays the exploitation of high-potential regions and limits the convergence speed.

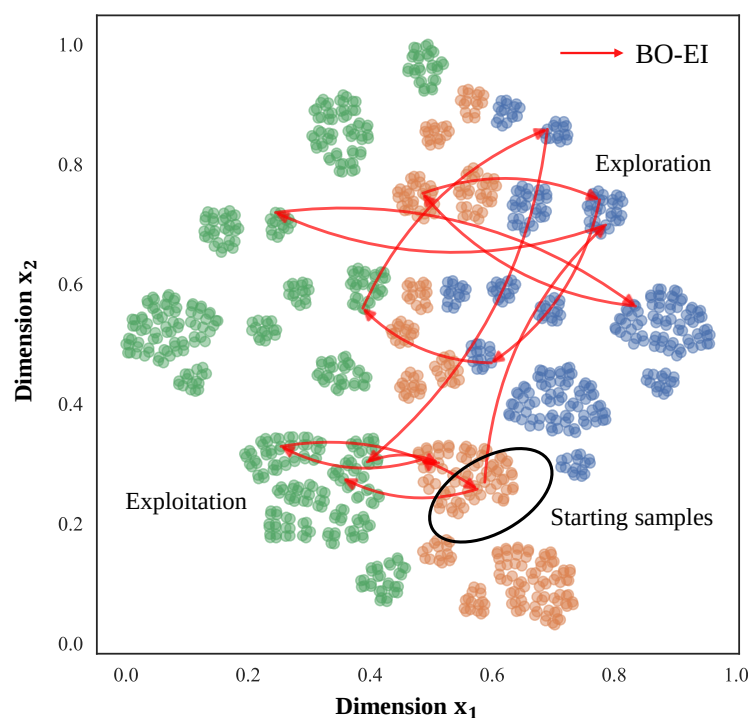


Figure 6. Using t -distributed stochastic neighbor embedding (t-SNE), the schematic illustrates the sequential BO in a high-dimensional design space with the standard EI strategy. The directed arrow represents the acquisition direction during iteration, while each cluster represents a group of samples with similarities via k-means clustering.

To overcome this, we first construct a sparse feature GP model [37] based on the initial samples $\mathcal{X}$ from the maximin criterion-based LHS method. The sparse GP model maps the original input data to a higher-dimensional feature space through a radial basis function (RBF) feature mapping function $\Phi(\cdot)$ to obtain sparsity. This mapping can produce more complex interactions in the data of discrete transistor sizing. An anisotropic squared

exponential (ASE) kernel is designed to capture the pairwise similarities between the mapped features in high-dimensional space, which can be denoted as

$$\kappa_{\text{ASE}}(\mathbf{x}, \mathbf{x}') = \exp \left[-\frac{1}{2} \sum_{i=1}^{d_x} \frac{(x_i - x'_i)^2}{l_i^2} \right]. \quad (30)$$

The ASE kernel allows different scales for each dimension to accommodate different degrees of smoothness in the data, where the i -th characteristic length-scale l_i can be solved by the Maximum Marginal Likelihood Estimation method. Given yield observations Z , the sparse GP model is trained by solving the linear system for the weights τ .

$$(K + \sigma_\epsilon^2 I) \tau = Z. \quad (31)$$

Using the sparse Cholesky decomposition, we can obtain the lower triangular matrix L , such that $K = LL^T$. This step is computationally efficient due to the sparsity of τ and avoids inverting the full covariance matrix. The predictive mean μ_* and variance Φ_* and variance σ_*^2 are computed using the posterior weights τ and the feature mapping of the new data point μ_* as follows

$$\begin{aligned} \mu_* &= \Phi_*^T \alpha \\ \sigma_*^2 &= k(\Phi_*, \Phi_*) - \Phi_*^T L^T L \Phi_*. \end{aligned} \quad (32)$$

Next, we utilize the model uncertainty of GP inference to encourage global exploration. Specifically, to evaluate the most promising and uncertain samples outside of the current observation, the posterior confidence intervals in each iteration t can be denoted as

$$\mathcal{I}_t(\mathbf{x}) = \left[\mu_{t-1}(\mathbf{x}) - \psi^{\frac{1}{2}} \sigma_{t-1}(\mathbf{x}), \quad \mu_{t-1}(\mathbf{x}) + \psi^{\frac{1}{2}} \sigma_{t-1}(\mathbf{x}) \right]. \quad (33)$$

For the multi-modal recognition of acquisition function mentioned in [38], the GP-UCB function $\mu(\mathbf{x}) + \psi^{\frac{1}{2}} \sigma(\mathbf{x})$ can derive samples with local optimums. We define a yield observation threshold h to help filter out local optimal points that satisfy $\mu(\mathbf{x}) < h$. The threshold is set to filter out those samples considered less profitable to implement yield analysis with a lower posterior mean.

Moreover, we reformulate an adaptive penalty function with an inverse quadratic penalty to prevent new observations from being trapped in local regions. The whole global exploration function can be denoted as

$$\mathcal{G}_t = \mu(\mathbf{x}) + \psi^{\frac{1}{2}} \sigma(\mathbf{x}) - \frac{\eta}{\|\mathbf{x} - \mathbf{x}_*\|_2^2}. \quad (34)$$

where the $\mathbf{x}_*$ denotes the present best solution. It requires a relatively larger η value to avoid being caught in local optima and probe more promising regions. Compared to using the wEI strategy Equation (15) in the global exploration stage, Equation (34) introduces the spatial information penalty using Euclidean distance between the new and previous samples to ensure sample diversity.

The multiple local optima of $\mathcal{G}_t$ are solved using the L-BFGS method. This heuristic method invokes multiple starting points, which should be sufficiently scattered to expect them to be near different peaks. For each initial point, L-BFGS gradually improves the acquisition value through an iterative process until convergence criteria are met, e.g., the Euclidean norm of the gradient is below a certain threshold, or the improvement of the function value is below a certain threshold.

4.2. Batch Sample Acquisition Strategy

We propose a batch sample acquisition strategy based on local optima of a sparse GP model, which aims for these two basic challenges summarized in [39]: knowledge gains from sample evaluation and sample diversity per batch, limiting the parallelism performance of optimization algorithms. In order to solve these two challenges, we will present how to select batch samples for evaluation in each iteration. The whole flow of the CBBO framework is shown in Algorithm 2.

Algorithm 2 Proposed CBBO method.

Require: Optimization budget T , batch size n_b .

Ensure: The global optimal solution $\mathbf{x}_*$.

- 1: Collect initial samples $\mathcal{X}$ using LHS methods.
 - 2: Build sparse GP model using $\mathcal{X}$.
 - 3: Obtain samples $\tilde{\mathbf{x}}_{1:k}$ from posterior distribution by maximizing Equation (35).
 - 4: **for** $t \leftarrow 1$ **to** T **do**
 - 5: **if** need exploration **then**
 - 6: Update the local optima $\tilde{\mathbf{x}}_{1:k}$ from sparse GP(θ_{t-1}).
 - 7: **end if**
 - 8: Obtain batch samples $\mathbf{x}_{t,1:n_b}$ using Equation (36).
 - 9: $\hat{\mathbf{z}}_{t,1:n_b} \leftarrow$ Implement yield analysis for $\mathbf{x}_{t,1:n_b}$.
 - 10: $\theta_t \leftarrow$ UpdateGP(θ_{t-1}).
 - 11: Augment the data $\mathcal{X} \leftarrow \mathcal{X} \cup \{\mathbf{x}_{t,1:n_b}, \hat{\mathbf{z}}_{t,1:n_b}\}$.
 - 12: **end for**
 - 13: The best yield observation $\mathbf{x}_* = \arg \max_{\mathbf{x} \in \mathcal{X}} \hat{\mathbf{z}}$.
-

For further exploitation in each iteration, we implement the batch sample collection and evaluation around the multiple local optimums $\tilde{\mathbf{X}}_{1:k}$ from the posterior distribution in Equation (34), referred to as the horizontal expansion of acquisition functions. To ensure sample diversity, each sample in $\tilde{\mathbf{X}}_{1:k}$ contributes to the sample collection. A penalized acquisition strategy is formulated based on the current posterior peaks to select a batch of samples that produces the maximum gain. In t -iteration, the collection of n_b batch samples can be represented as

$$\mathbf{x}_{t,i} = \arg \max_{\mathbf{x} \in \mathcal{D}} \left\{ \alpha(\mathbf{x}; \theta_{t-1}) \prod_{j=1}^k \varphi(\mathbf{x}; \tilde{\mathbf{x}}_j) \right\}, 1 \leq i \leq n_b. \quad (35)$$

The $\alpha(\mathbf{x}; \theta_{t-1})$ denotes the weighted EI function according to the posterior distribution

$$\alpha(\mathbf{x}; \theta_{t-1}) = \text{PF}(\mathbf{x}) \cdot \text{EI}(\mathbf{x}). \quad (36)$$

Moreover, $\varphi(\mathbf{x}; \tilde{\mathbf{x}}_j)$ is a pre-defined local penalty function effected at $\tilde{\mathbf{x}}_j$. To encourage exploration of unknown regions, the penalty function $\varphi(\cdot)$ should constrain the search space to exclude samples nearby $\tilde{\mathbf{x}}_j$ by reducing the acquisition value. In this problem, $\varphi(\cdot)$ is defined based on the posterior mean μ_n and variance σ_n^2

$$\varphi(\mathbf{x}; \tilde{\mathbf{x}}_j) = \frac{1}{2} \cdot \text{erfc} \left(-\frac{L \cdot \|\mathbf{x} - \tilde{\mathbf{x}}_j\| - (M - \mu_n(\tilde{\mathbf{x}}_j))}{\sqrt{2} \cdot \sigma_n(\tilde{\mathbf{x}}_j)} \right), \quad (37)$$

where L is a valid constant. To avoid excessive concentration near known samples, $\text{erfc}(\cdot)$ denotes the complementary error function for emulating probability distributions in the neighborhood of observed samples, which can reflect the belief about the improvement from $\mu_n(\tilde{\mathbf{x}}_j)$ to the theoretical maximum yield value 1. For a sample $\mathbf{x}$ around $\tilde{\mathbf{x}}_j$, $\text{erfc}(\cdot)$ will have a smaller value that excludes samples near $\tilde{\mathbf{x}}_j$ to reduce the attractiveness of exploring

these regions. Moreover, when $\sigma_n^2(\mathbf{x})$ has a smaller value, $\varphi(\cdot)$ will produce a wider value correction, which prefers samples with less estimation uncertainty to maintain an efficient search for the global optimum.

The acquisition function in Equation (35) is solved using an iteration strategy. In each iteration, each batch sample selected will invoke the penalization until n_b samples are collected. The global exploration phase continues until the regions with high yield values are fully explored, i.e., when the variance among the local optimums meets $\sigma_n^2 < \epsilon$. Intuitively, this happens when we have learned the yield objective functions with high precision. The proposed framework alternates between two phases using EI-based local exploitation and uncertainty-based global exploration. Compared to conventional BO methods with the computational complexity from matrix inverse operation $\mathcal{O}(N^3)$, the complexity of sparse Cholesky decomposition is highly related to the sparsity. With k non-zero elements ($k \ll n^2$), the complexity can be approximated as $\mathcal{O}(k \cdot \log(k))$. The actual efficiency also hinges on storage formats and recursive strategies.

5. Experimental Results

In this section, we demonstrate the accuracy and efficiency of the proposed optimization framework CBBO using SRAM column and amplifier circuits. We selected benchmark circuits that require high-dimensional yield analysis and operate within large-scale design optimization spaces. We implemented several SOTA optimization methods, such as WEIBO [22], FTBO [23], and AS AIS [17] for efficiency comparison. The selection of design variables is usually restricted to a range of discrete integers, which is highly related to the circuit type. CBBO was configured to evaluate a batch of $n_b = 6$ designs in parallel. We implemented sparse matrix computations on large-scale datasets using the library `sksparse.cholmod` [40].

To evaluate the robustness and efficiency, we implemented five tests for each method in the yield optimization experiments and utilized different random seeds to initialize algorithms and data. Corresponding yield estimation results of solved optimal design solutions $\mathbf{x}^*$ were evaluated via brute MC methods with 5×10^4 simulations. All experiments were conducted on a Linux server with 128GB RAM and 4 NVIDIA Tesla V100 GPUs.

5.1. Experiments on SRAM Column

As shown in Figure 7, a 32-bit SRAM column contains 32 replicated bitcells. In a single SRAM bitcell, transistors consisting of M1, M5, M2, and M6 form two cross-coupled inverters and two steady states, while M3 and M4 control access to the storage cell during read and write operations. To construct the highest leakage current condition for the dynamic read operation, we stored “0” in the first bitcell and the opposite state in the remaining bitcells. During the read operation, the bitline voltage V_{BL} decreases through pull-down transistors after the wordline enables the access transistors, and the voltage difference between two bitlines can be captured by a sense amplifier. Without loss of generality, we took read delay T_{read} and bitline discharge current I_{read} as the read failure mechanisms of interest, which can be obtained from the transient response of the circuits. The design of experiments (DoE) was implemented on a 28 nm CMOS process library. Moreover, mismatches containing threshold voltage v_{th0} , offset voltage v_{off} , and low-field mobility μ_0 were selected and modeled as independent Gaussian random variables. The 32-bit SRAM column has a total of 576 independent Gaussian random variables. For the optimization problem, due to the symmetry structure of SRAM bitcell, we considered three design parameters containing the width parameters of access, pull-up, and pull-down transistors. The read delay should satisfy $T_{read} \leq 1.5$ ns under supply voltage $V_{dd} = 0.8$ V for nominal specification requirements.

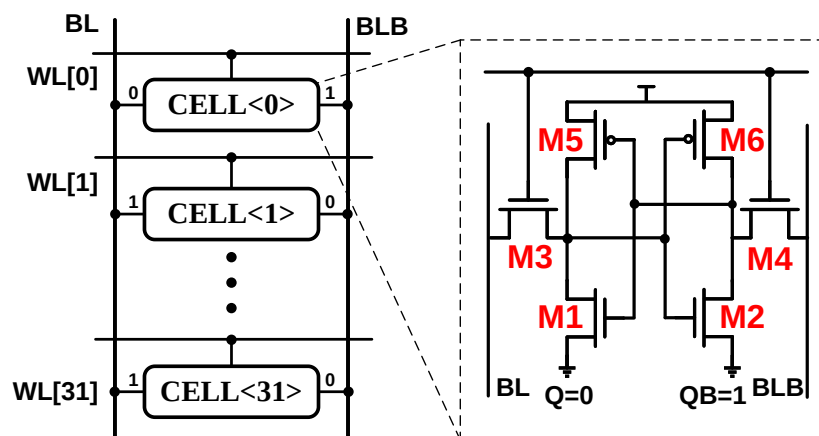


Figure 7. The simplified schematic of a 32-bit SRAM column.

We selected $\{\mathbf{x}_i^s\}_{i=1}^5$ with 800 training samples to generate source models; the model transfer stage involved 200 training samples for each target design $\mathbf{x}^t$. To validate the efficiency of transfer learning, we selected the fine-tune-all method as a baseline, which transferred the whole network parameters from pre-trained source models to adapt to limited labeled target data. Additionally, we set up two different loss functions $\mathcal{L}_1 = \mathcal{L}_{cls} + \lambda \mathcal{L}_{wmmd}$ and $\mathcal{L}_2 = \mathcal{L}_{cls} + \lambda \mathcal{L}_{mmd}$ for the ablation study with the same training set. To prevent the risk of overfitting issues in some tasks, we employed the random search method to tune trade-off parameters λ and β . Value ranges of λ and β were 0.01 to 0.5, and 0.001 to 0.1, respectively. Then, the backpropagation was implemented with the Adam optimizer. We also prepared several equal-sized labeled sample sets to validate the fine-tuning process. One set was used for training, and the remaining sets were used for testing.

As shown in Figure 8, we present the classification performance of the CNN model accuracy with the number of training epochs and folds in the fine-tuning process. We utilize the accuracy index as the evaluation metric, which represents the proportion of correctly classified samples among all test samples:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}. \quad (38)$$

We chose samples near the failure boundary to form the test set, to eliminate the misrepresentation of the accuracy index for imbalanced data. The proposed boundary-aware transfer strategy achieves better prediction accuracy without retraining the entire network. Figure 8b demonstrates that increasing the number of training folds enhances model generalization, as reflected in improved classification accuracy. The entire model transfer process can achieve similar performance as the fine-tune all method by using only a modest number of samples. In contrast, other methods require a large number of training iterations to converge. Moreover, we show the trend in model estimation accuracy as the number of folds increases for the cross-validation.

For the 32-bit SRAM column, we selected two different designs and implemented the comparison of the failure probability and FoM convergence of the DTL-IS with several SOTA yield estimation methods (see Figure 9). To guarantee accuracy and reliability, we set the FoM threshold as $\rho_{th} = 0.1$, which reflects that the failure rate estimation reaches 90% accuracy level with 90% confidence interval. The failure rate estimation of the AS AIS method [17] was implemented using the sum of the failure rate of each performance of interest. The proposed DTL-IS algorithm provides a relatively fast convergence speed, exhibiting a $600 \sim 850 \times$ speedup over the MC method, which serves as the golden standard. Moreover, DTL-IS improves the estimation accuracy with HSPICE simulation-assisted calibration

of the proposal distribution, where the relative error is no more than 3.5% with an average speedup of $12.53\times$ and $4.26\times$ compared to MFRIS [15] and ASDK [20], respectively.

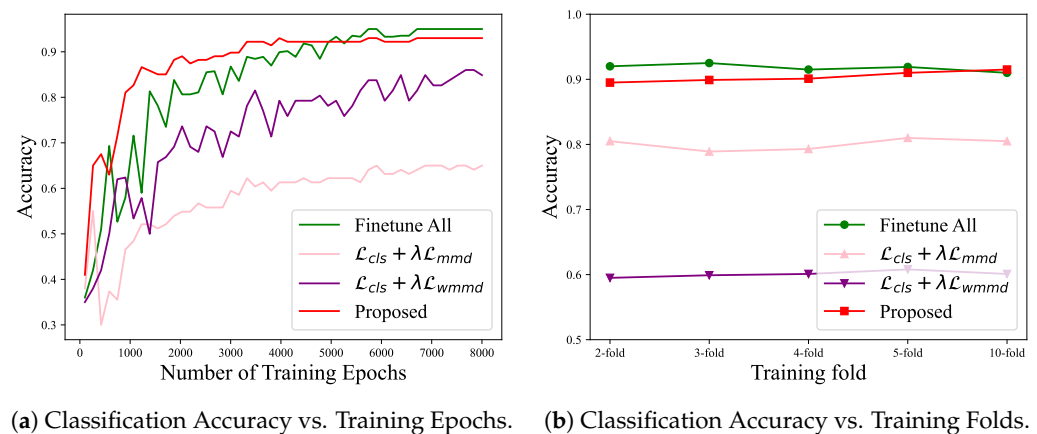


Figure 8. Convergence of classification accuracy during the model fine-tuning process for the 32-bit SRAM column.

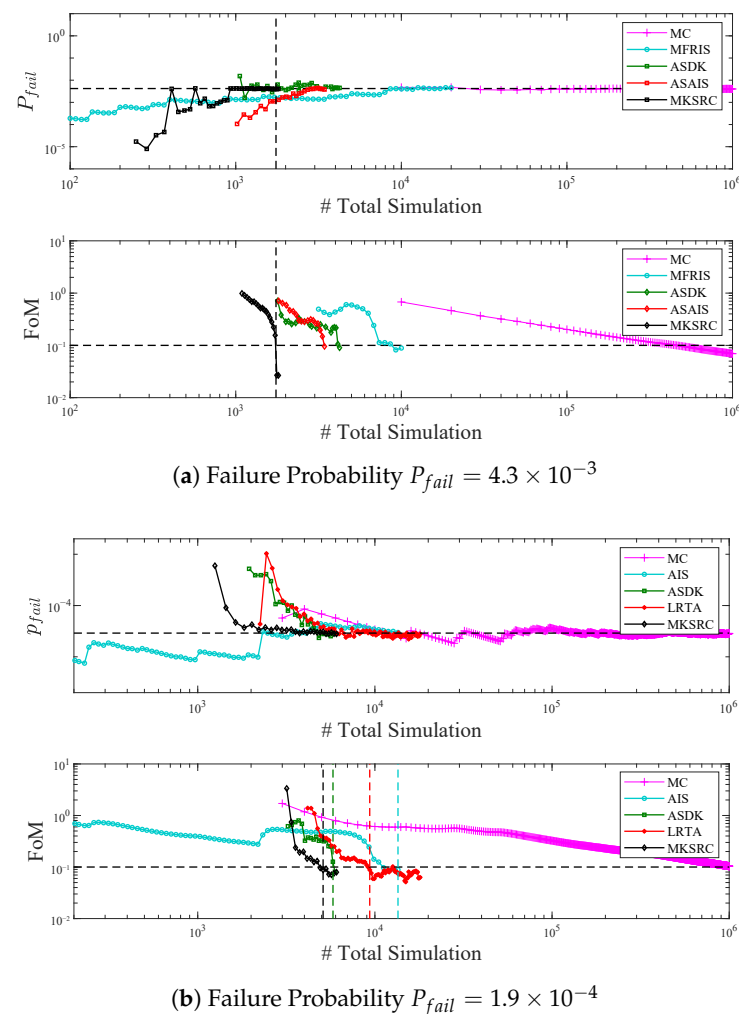


Figure 9. Comparison of the convergence of failure probability and figure of merit (FoM) for different design samples in a 32-bit SRAM column under varying yield levels.

The final optimization results and simulation overheads are shown in Table 2. We consider the average number of simulations for each method as a reflection of its convergence speed. It can be concluded that high-yield design solutions can be obtained from the best

observation of EI-based optimization methods. With the benefit of HSPICE calibration, CBBO outperforms sensitivity-based IS methods with an improvement in mean performance. In contrast, the optimization results of AS AIS are prone to large fluctuations in high-dimensional scenarios. As for stability and robustness, CBBO introduces the penalized acquisition mechanism to simultaneously optimize samples on the batch, which brings a 0.01–0.64% variance induction.

Table 2. Optimization results and the total simulation overheads for the 32-bit SRAM column.

Methods		WEIBO	FTBO	AS AIS	CBBO
Yield	<i>Best</i>	99.92%	99.995%	99.93%	99.998%
	<i>Worst</i>	99.55%	99.88%	98.45%	99.86%
	<i>Mean</i>	99.74%	99.93%	99.19%	99.93%
	<i>Std. Dev</i>	0.19%	0.06%	0.74%	0.07%
Total #sim.	<i>Best</i>	118,500	75,310	56,190	27,260
	<i>Worst</i>	168,400	87,740	69,630	34,690
	<i>Mean</i>	143,450	81,525	62,910	30,975
Runtime (min.)	<i>Best</i>	2848	1808	1348	654
	<i>Worst</i>	4042	2106	1671	833
	<i>Mean</i>	3443	1957	1510	743
Speedup (#sim.)		1	1.75	2.28	4.63

5.2. Experiments on a Three-Stage Amplifier

The simplified schematic of a three-stage operational amplifier consisting of a reversed nested Miller compensation structure is shown in Figure 10. For low-supply voltage applications, the symmetrical structure requires the same design parameters for M1 and M2; M3 and M4; M5, M7, and M10. The load capacitance C_L is set to 500 pF. The design parameters contain the width parameters of each transistor, and compensation capacitors (C_1 and C_2) and resistor R_C , where a total of 11 design parameters are optimized using a 40 nm process library. In the detailed experimental settings, we consider four performance specifications containing DC gain, gain-bandwidth product, phase margin, and common-mode rejection ratio (CMRR), where the nominal specifications should satisfy

$$\begin{cases} \text{gain} \geq 50 \text{ dB} \\ \text{GBW} \geq 2.5 \text{ MHz} \\ \text{PM} \geq 50^\circ \\ \text{CMRR} \geq 60 \text{ dB.} \end{cases} \quad (39)$$

For each collected sample, a nominal constraint function $\text{PF}(\mathbf{x})$ is first applied to verify whether the specified performance specifications are met. Only when all nominal constraints are met can the subsequent failure probability analysis be performed.

The estimation of P_{fail} for the three-stage op-amp involves various failure mechanisms. Each transistor is subject to the same three variation parameters as in the SRAM column experiment, resulting in a total of 36 variation parameters. According to the circuit size, we select $\{\mathbf{x}_i^s\}_{i=1}^8$ with 300 variation samples to train source models, and the convergence of model accuracy with the number of training epochs and folds in the finetuning process is shown in Figure 11.

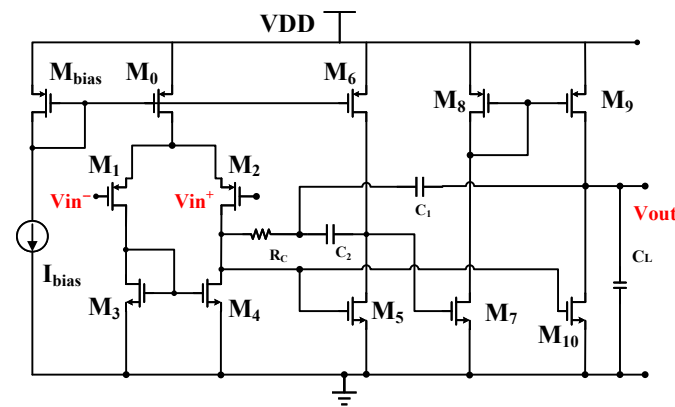
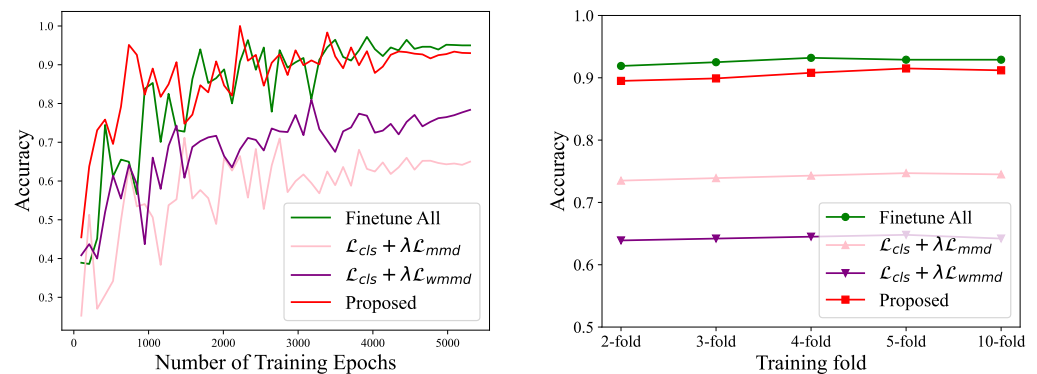


Figure 10. The schematic of a three-stage amplifier circuit.



(a) Classification Accuracy vs. Training Epochs. **(b)** Classification Accuracy vs. Training Folds.

Figure 11. Convergence of classification accuracy during the model fine-tuning process for the three-stage amplifier.

As the optimization results in Table 3, the advantages of the CBBO algorithm are more prominent in a high-dimensional search space. CBBO reduces the simulation overhead in the optimization process by reusing prior knowledge from source models. By contrast, FTBO gradually improves the analysis accuracy in the higher yield region, while elaborate yield analysis in the lower yield region will not be implemented. Both of these two methods improve the form of yield analysis and perform batch simulations at each iteration to accelerate the convergence of the optimization process.

Table 3. Optimization results and the total simulation overheads for the three-stage amplifier.

Methods		WEIBO	FTBO	AS AIS	CBBO
Yield	Best	99.82%	99.95%	99.85%	99.92%
	Worst	98.2%	99.86%	98.45%	99.8%
	Mean	99.01%	99.91%	99.15%	99.86%
	Std. Dev	0.81%	0.05%	0.7%	0.06%
Total #sim.	Best	41,820	24,720	36,190	24,105
	Worst	74,636	33,960	62,630	33,265
	Mean	58,228	29,340	49,410	28,685
Runtime (min.)	Best	1183	921	762	483
	Worst	1492	1122	982	652
	Mean	1337	1022	872	562
Speedup (#sim.)		1	1.98	1.18	2.03

5.3. Experiments on Rail-to-Rail Amplifier

To provide sufficient voltage margin in low-supply voltage scenarios, the simplified schematic of a rail-to-rail operational amplifier is shown in Figure 12. The MOSFETs M1, M2, M3, and M6 form the main stage of the NMOS input level, while M9, M12, M13, and M14 constitute the main stage of the PMOS input level. M4, M5, and M7 form the slave stage of the NMOS input level, and M8, M10, and M11 form the slave stage of the PMOS input level. Moreover, M6 operates in the linear region to save voltage margin. M21 and M22 use a substrate bias structure to form the second-stage output level, achieving an output swing that is approximately rail-to-rail. The supply voltage is set as $V_{dd} = 0.65$ V. Using a 40 nm process library, the nominal specifications should satisfy

$$\begin{cases} \text{gain} \geq 80 \text{ dB} \\ \text{GBW} \geq 6.5 \text{ MHz} \\ \text{PM} \geq 60^\circ \\ P_{\text{total}} \leq 0.3 \text{ mW}. \end{cases} \quad (40)$$

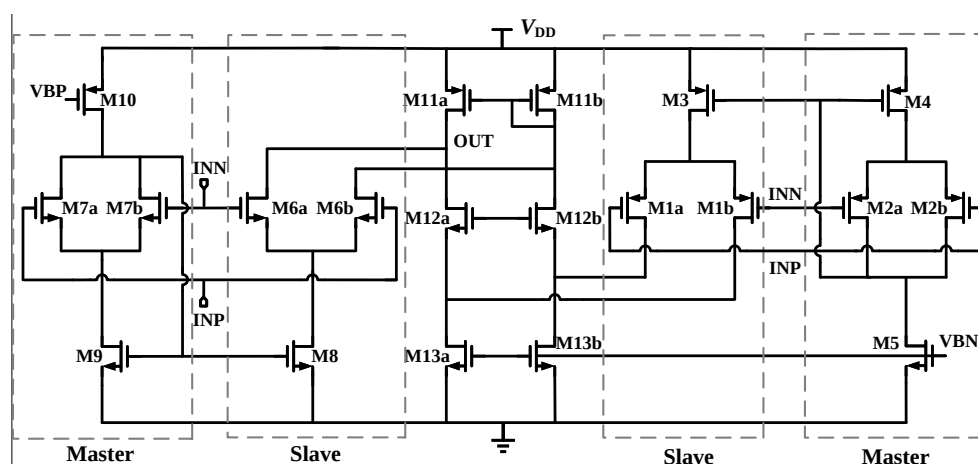
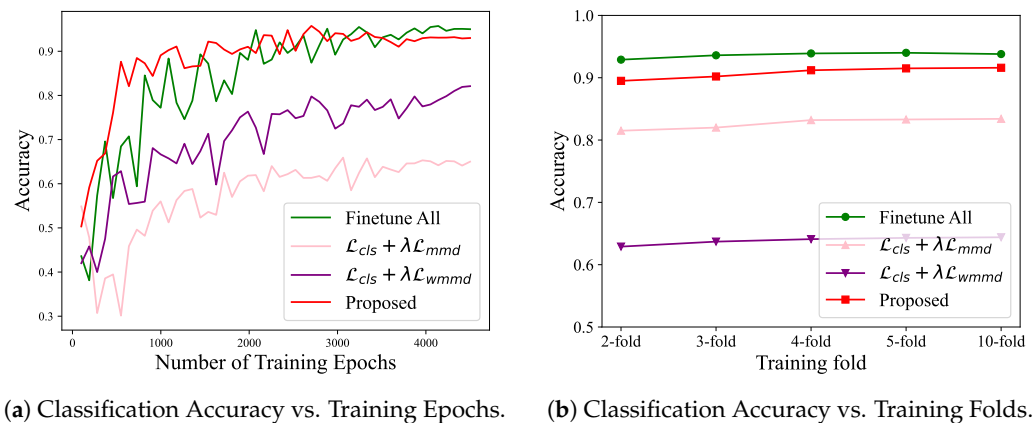


Figure 12. The schematic of a low-power rail-to-rail op-amp circuit.

In the detailed optimization problem, the rail-to-rail amplifier circuit involves 22 MOS transistors and C0, R0 in total. In terms of circuit design, we concentrate on two particular design variables: the width and length of an individual transistor. Due to the larger circuit dimensions, we train source models $\{\mathbf{x}_i^s\}_{i=1}^{10}$ using 300 variation samples, where the convergence of accuracy is shown in Figure 13.

The experimental results for yield optimization are presented in Table 4. Compared to the SRAM column and three-stage amplifier, the yield analysis for the rail-to-rail amplifier has some degree of accuracy loss, due to the more complex nonlinear mapping of circuit performance to process parameters in the low-voltage domain. To improve accuracy, DTL-IS can help learn the nonlinear mapping and iteratively adjust the mixture proposal distribution according to the estimated variance. The results show that WEIBO fails to find the optimal design with sufficient iterations, while CBBO can tolerate more iterations due to lower simulation overhead and ensure accurate yield analysis. In the detailed optimization method, the penalty mechanism can help improve the convergence speed by reducing the exploration of unnecessary regions by creating exclusion regions around evaluated samples in the larger design space.



(a) Classification Accuracy vs. Training Epochs. (b) Classification Accuracy vs. Training Folds.

Figure 13. Convergence of classification accuracy during the model fine-tuning process for the rail-to-rail op-amp.

Table 4. Optimization results and the total simulation overheads for the rail-to-rail amplifier.

Methods		WEIBO	FTBO	AS AIS	CBBO
Yield	Best	98.22%	99.45%	99.25%	99.52%
	Worst	95.5%	99.13%	97.06%	99.06%
	Mean	96.82%	98.81%	98.2%	99.29%
	Std. Dev	1.32%	0.32%	1.05%	0.23%
Total #sim.	Best	61,455	27,080	32,190	27,860
	Worst	78,221	36,425	54,350	31,200
	Mean	69,838	31,752	43,270	29,530
Runtime (min.)	Best	1321	1084	912	621
	Worst	1785	1383	1211	892
	Mean	1552	1233	1061	756
Speedup (#sim.)		1	2.19	1.61	2.36

5.4. Computational Cost Analysis

As for the CBBO framework, we further analyze the distribution of simulation cost across different stages for the three benchmark circuits, as shown in Figure 14. Over the complete optimization process, the dominant portion of simulation resources is spent on failure rate estimation based on distorted sampling distributions, while the calibration of OSV is negligible. This observation indicates that the overall runtime is mainly determined by iterative yield estimation, rather than by fine-tuning the CNN models and model inference.

In the proposed optimization framework, the additional computational overhead introduced by DTL is relatively small. Specifically, the pre-trained CNN models can be reused across multiple design iterations, thus minimizing the need for extensive retraining. During optimization, fine-tuning is performed on a limited target dataset and typically converges within a few epochs. Adversarial gradient calculations for calibration are limited to a small subset of boundary samples. As a result, the combined cost of training and fine-tuning constitutes only a minor portion of the total runtime, with SPICE simulations accounting for the vast majority of the computational budget, often by one to two orders of magnitude.

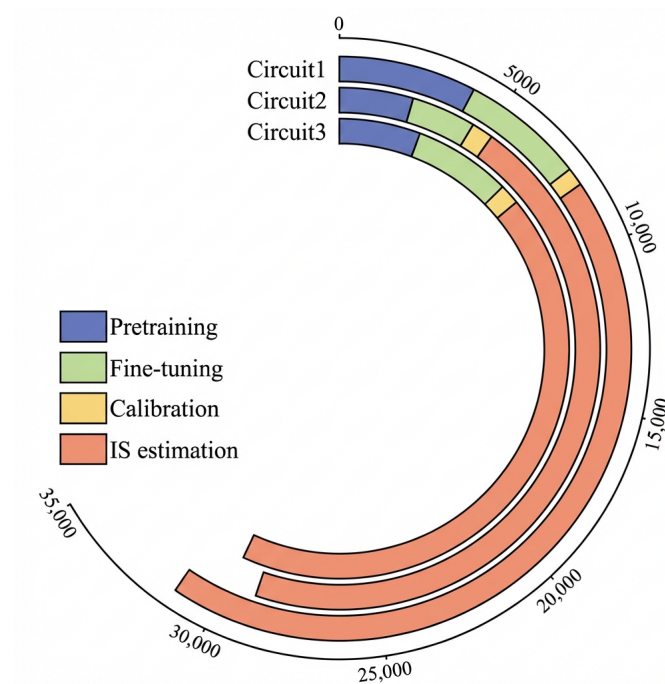


Figure 14. Distribution of simulation counts across different stages of the CBBO optimization process.

6. Conclusions

In this article, we propose a cascaded batch optimization framework equipped with deep transfer learning that leverages prior knowledge to reduce simulation overhead in the whole yield optimization. Given pre-trained CNN models, the failure probability estimation of unseen samples can be significantly accelerated. Then, a novel penalization-improvement acquisition strategy is formulated to select batch samples in the feasible domain to further improve the quality of solutions. Compared with the state-of-the-art methods, the experimental results demonstrated that the proposed method can gain a $2.02\times$ – $4.63\times$ speedup without loss of accuracy and facilitate the multiple failure regions for analog circuits. Future work could focus on extending batch optimization to handle asynchronous parallel evaluation and multi-fidelity modeling, further improving scalability and efficiency in yield-aware analog circuit design.

Author Contributions: Conceptualization, Z.W.; methodology, Z.W.; software, Z.W.; validation, Z.W. and K.S.; formal analysis, K.S.; investigation, K.S.; resources, X.S.; data curation, X.S.; writing—original draft preparation, Z.W. and K.S.; writing—review and editing, Z.W. and K.S.; visualization, Z.W. and K.S.; supervision, X.S.; project administration, X.S.; funding acquisition, X.S. All authors have read and agreed to the published version of the manuscript.

Funding: This research was partly funded by the National Natural Science Foundation of China under Grant 62274034, and funded by the Natural Science Foundation of Jiangsu Province (BK20232005). We thank the National Center of Technology Innovation for EDA for the hardware support.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Cho, K.; Choi, H.; Jung, I.J.; Oh, J.; Oh, T.W.; Kim, K.; Kim, G.; Choi, T.; Sim, C.; Song, T.; et al. SRAM write- and performance-assist cells for reducing interconnect resistance effects increased with technology scaling. *IEEE J.-Solid-State Circuits* **2022**, *57*, 1039–1048. [\[CrossRef\]](#)
2. Lee, I.; Jeong, H.; Baeck, S.; Gupta, S.; Park, C.; Seo, D.; Choi, J.; Kim, J.; Kim, H.; Kang, J.; et al. 24.3 A voltage and temperature tracking SRAM assist supporting 740mV dual-rail offset for low-power and high-performance applications in 7nm EUV FinFET technology. In Proceedings of the 2019 IEEE International Solid-State Circuits Conference—(ISSCC), San Francisco, CA, USA, 17–21 February 2019; pp. 392–394. [\[CrossRef\]](#)
3. Sohn, S. Robust parameter design for integrated circuit fabrication procedure with respect to categorical characteristic. *Reliab. Eng. Syst. Saf.* **1999**, *66*, 253–260. [\[CrossRef\]](#)
4. Zhao, Z.; Wang, J.; Tao, Q.; Li, A.; Chen, Y. An unknown wafer surface defect detection approach based on incremental learning for reliability analysis. *Reliab. Eng. Syst. Saf.* **2024**, *244*, 109966. [\[CrossRef\]](#)
5. Mukhopadhyay, S.; Mahmoodi, H.; Roy, K. Modeling of failure probability and statistical design of SRAM array for yield enhancement in nanoscaled CMOS. *IEEE Trans.-Comput.-Aided Des. Integr. Circuits Syst.* **2005**, *24*, 1859–1880. [\[CrossRef\]](#)
6. Zhang, A.; Huang, C.; Guo, T.; Chen, A.; Guo, S.; Wang, R.; Huang, R.; Xie, J. Reliability variability simulation methodology for IC design: An EDA perspective. In Proceedings of the 2015 IEEE International Electron Devices Meeting (IEDM), Washington, DC, USA, 7–9 December 2015; pp. 11.5.1–11.5.4. [\[CrossRef\]](#)
7. Singhee, A.; Rutenbar, R.A. Why quasi-Monte Carlo is better than Monte Carlo or Latin hypercube sampling for statistical circuit analysis. *IEEE Trans.-Comput.-Aided Des. Integr. Circuits Syst.* **2010**, *29*, 1763–1776. [\[CrossRef\]](#)
8. Liakos, K.G.; Plessas, F. Analog design and machine learning: A review. *Electronics* **2025**, *14*, 3541. [\[CrossRef\]](#)
9. Touloupas, K.; Sotiriadis, P.P. Mixed-variable Bayesian optimization for analog circuit sizing through device representation learning. *Electronics* **2022**, *11*, 3127. [\[CrossRef\]](#)
10. Isa, A.A.H.; Sulthan, S.M.; Dani, M.N.; Tan, S.J. Enhanced lithium-ion battery state-of-charge estimation via Akima–Savitzky–Golay OCV-SOC mapping reconstruction and Bayesian-optimized adaptive extended Kalman filter. *Energies* **2025**, *18*, 6192. [\[CrossRef\]](#)
11. Gupta, S.; Calhoun, B.H. Dynamic read Vmin and yield estimation for nanoscale SRAMs. *IEEE Trans. Circuits Syst. I Regul. Pap.* **2021**, *68*, 1171–1182. [\[CrossRef\]](#)
12. Yao, J.; Ye, Z.; Wang, Y. An efficient SRAM yield analysis and optimization method with adaptive online surrogate modeling. *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* **2015**, *23*, 1245–1253. [\[CrossRef\]](#)
13. Shi, X.; Liu, F.; Yang, J.; He, L. A fast and robust failure analysis of memory circuits using adaptive importance sampling method. In Proceedings of the 55th Annual Design Automation Conference (DAC '18), San Francisco, CA, USA, 24–29 June 2018. [\[CrossRef\]](#)
14. Shi, X.; Yan, H.; Zhang, J.; Huang, Q.; Shi, L.; He, L. Efficient yield analysis for SRAM and analog circuits using meta-model based importance sampling method. In Proceedings of the 2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD), Westminster, CO, USA, 4–7 November 2019; pp. 1–8. [\[CrossRef\]](#)
15. Wang, M.; Yan, C.; Li, X.; Zhou, D.; Zeng, X. High-dimensional and multiple-failure-region importance sampling for SRAM yield analysis. *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* **2017**, *25*, 806–819. [\[CrossRef\]](#)
16. Liu, Y.; Dai, G.; Cheng, Y.; Kang, W.; Xing, W.W. Opt: Optimal proposal transfer for efficient yield optimization for analog and SRAM circuits. In Proceedings of the 2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD), San Francisco, CA, USA, 28 October–2 November 2023; pp. 1–9. [\[CrossRef\]](#)
17. Hu, W.; Wang, Z.; Yin, S.; Ye, Z.; Wang, Y. Sensitivity importance sampling yield analysis and optimization for high sigma failure rate estimation. In Proceedings of the 2021 58th ACM/IEEE Design Automation Conference (DAC), San Francisco, CA, USA, 5–9 December 2021; pp. 895–900. [\[CrossRef\]](#)
18. Shi, X.; Yan, H.; Huang, Q.; Xuan, C.; He, L.; Shi, L. A compact high-dimensional yield analysis method using low-rank tensor approximation. *ACM Trans. Des. Autom. Electron. Syst.* **2022**, *27*, 19. [\[CrossRef\]](#)
19. Xu, H.; Fan, H.; Jiang, B.; Chen, J.; Tong, Q.; Zou, X. ML-accelerated yield analysis framework using regularization for sparsity in high-sigma and high-dimensional scenarios. *IEEE Trans.-Comput.-Aided Des. Integr. Circuits Syst.* **2023**, *42*, 1161–1170. [\[CrossRef\]](#)
20. Yin, S.; Dai, G.; Xing, W.W. High-dimensional yield estimation using shrinkage deep features and maximization of integral entropy reduction. In Proceedings of the 2023 28th Asia and South Pacific Design Automation Conference (ASP-DAC), Tokyo, Japan, 16–19 January 2023; pp. 283–289.
21. Shahriari, B.; Swersky, K.; Wang, Z.; Adams, R.P.; de Freitas, N. Taking the human out of the loop: A review of Bayesian optimization. *Proc. IEEE* **2016**, *104*, 148–175. [\[CrossRef\]](#)
22. Wang, M.; Lv, W.; Yang, F.; Yan, C.; Cai, W.; Zhou, D.; Zeng, X. Efficient yield optimization for analog and SRAM circuits via Gaussian process regression and adaptive yield estimation. *IEEE Trans.-Comput.-Aided Des. Integr. Circuits Syst.* **2018**, *37*, 1929–1942. [\[CrossRef\]](#)

23. Wang, X.; Yan, C.; Ma, Y.; Yu, B.; Yang, F.; Zhou, D.; Zeng, X. Analog circuit yield optimization via freeze-thaw Bayesian optimization technique. *IEEE Trans.-Comput.-Aided Des. Integr. Circuits Syst.* **2022**, *41*, 4887–4900. [\[CrossRef\]](#)
24. Davalos-Guzman, J.; Chavez-Hurtado, J.L.; Brito-Brito, Z. Accelerating high-frequency circuit optimization using machine learning-generated inverse maps for enhanced space mapping. *Electronics* **2025**, *14*, 3097. [\[CrossRef\]](#)
25. Seeger, M.W. Gaussian processes for machine learning. *Int. J. Neural Syst.* **2004**, *14*, 69–106. [\[CrossRef\]](#)
26. Jones, D.R.; Schonlau, M.; Welch, W.J. Efficient global optimization of expensive black-box functions. *J. Glob. Optim.* **1998**, *13*, 455–492. [\[CrossRef\]](#)
27. Gardner, J.R.; Kusner, M.J.; Xu, Z.E.; Weinberger, K.Q.; Cunningham, J.P. Bayesian optimization with inequality constraints. *Int. Conf. Mach. Learn. (ICML)* **2014**, *2014*, 937–945.
28. He, H.; Garcia, E.A. Learning from imbalanced data. *IEEE Trans. Knowl. Data Eng.* **2009**, *21*, 1263–1284. [\[CrossRef\]](#)
29. Ioffe, S.; Szegedy, C. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In Proceedings of the 32nd International Conference on Machine Learning (ICML '15), Lille, France, 6–11 July 2015; Volume 37, pp. 448–456.
30. Saito, K.; Kim, D.; Sclaroff, S.; Darrell, T.; Saenko, K. Semi-supervised domain adaptation via minimax entropy. In Proceedings of the 2019 IEEE/CVF International Conference on Computer Vision (ICCV), Seoul, Republic of Korea, 27 October–2 November 2019. [\[CrossRef\]](#)
31. Piersigilli, P.; Citroni, R.; Mangini, F.; Frezza, F. Electromagnetic Techniques Applied to Cultural Heritage Diagnosis: State of the Art and Future Prospective: A Comprehensive Review. *Appl. Sci.* **2025**, *15*, 6402. [\[CrossRef\]](#)
32. Gao, J.; Ge, L.; Li, K.; Ngo, H.Q.; Zhang, A. On handling negative transfer and imbalanced distributions in multiple source transfer learning. *Stat. Anal. Data Min. Asa Data Sci. J.* **2014**, *7*, 254–271.
33. Yosinski, J.; Clune, J.; Bengio, Y.; Lipson, H. How transferable are features in deep neural networks? In Proceedings of the 27th International Conference on Neural Information Processing Systems (NIPS '14), Montreal, QC, Canada, 8–13 December 2014; Volume 2, pp. 3320–3328.
34. Kingma, D.P.; Ba, J. Adam: A method for stochastic optimization. *arXiv* **2015**, arXiv:1412.6980. [\[CrossRef\]](#)
35. Yang, Y.; Khanna, R.; Yu, Y.; Gholami, A.; Keutzer, K.; Gonzalez, J.E.; Ramchandran, K.; Mahoney, M.W. Boundary thickness and robustness in learning models. In Proceedings of the 34th International Conference on Neural Information Processing Systems, NIPS '20, Virtual, 6–12 December 2020; Curran Associates Inc.: Red Hook, NY, USA, 2020.
36. Poloczek, M.; Wang, J.; Frazier, P.I. Warm starting bayesian optimization. In Proceedings of the 2016 Winter Simulation Conference, WSC '16, Arlington, VA, USA, 11–14 December 2016; pp. 770–781.
37. Leibfried, F.; Dutordoir, V.; John, S.; Durrande, N. A tutorial on sparse gaussian processes and variational inference. *arXiv* **2022**, arXiv:2012.13962. [\[CrossRef\]](#)
38. Gu, T.; Li, W.; Zhao, A.; Bi, Z.; Li, X.; Yang, F.; Yan, C.; Hu, W.; Zhou, D.; Cui, T.; et al. Bbgp-sdfo: Batch bayesian and gaussian process enhanced subspace derivative free optimization for high-dimensional analog circuit synthesis. *IEEE Trans.-Comput.-Aided Des. Integr. Circuits Syst.* **2024**, *43*, 417–430. [\[CrossRef\]](#)
39. Zhang, S.; Yang, F.; Yan, C.; Zhou, D.; Zeng, X. An efficient batch-constrained bayesian optimization approach for analog circuit synthesis via multiobjective acquisition ensemble. *IEEE Trans.-Comput.-Aided Des. Integr. Circuits Syst.* **2022**, *41*, 1–14. [\[CrossRef\]](#)
40. Sparse Cholesky Decomposition Package, v0.4.4. Available online: <https://scikit-sparse.readthedocs.io/en/latest/> (accessed on 21 January 2026).

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.